**Ministerul Educaţiei şi Cercetării al Republicii Moldova**
**Universitatea Tehnică a Moldovei**
**Facultatea Calculatoare, Informatică și Microelectronică**

# Laboratory work 4
# Study the dynamic programming method of designing algorithms.

Elaborated:
st. gr. FAF-242 Brînză Vasile

Verified:
asist. univ. Fiștic Cristofor

Chişinău - 2026

# TABLE OF CONTENTS

# ALGORITHM ANALYSIS

## Objective

To study the dynamic programming method of designing algorithms.

## Tasks

1. To study the dynamic programming method of designing algorithms;
2. To implement in a programming language algorithms Dijkstra and Floyd–Warshall using dynamic programming;
3. Do empirical analysis of these algorithms for a sparse graph and for a dense graph.
4. Increase the number of nodes in graphs and analyze how this influences the algorithms. Make a graphical presentation of the data obtained;
5. To make a report.

## Theoretical Notes

Dynamic programming is a computer programming technique where an algorithmic problem is first broken down into sub-problems, the results are saved, and then the sub-problems are optimized to find the overall solution. Unlike simple recursion, dynamic programming avoids redundant computation by storing intermediate results, either using a top-down approach with memoization or a bottom-up approach with tabulation. It is especially effective for optimization problems, such as finding the shortest path between nodes in a graph.

Dijkstra's Algorithm finds the shortest path from a single source node to all other nodes in a weighted graph with non-negative edge weights. It works greedily by always expanding the unvisited node with the smallest known distance. Using a min-heap (priority queue), it runs in $O((V + E) \log V)$ time, where V is the number of vertices and E is the number of edges. Dijkstra is well suited for sparse graphs, where E is much smaller than $V^2$, and must be run V times to solve the all-pairs shortest path problem.

The Floyd-Warshall Algorithm solves the all-pairs shortest path problem directly, finding the shortest paths between every pair of nodes in a single execution. It uses a dynamic programming approach by iteratively improving path estimates through each intermediate node. Its time complexity is $O(V^3)$ and space complexity is $O(V^2)$, making it straightforward to implement but expensive for large graphs. Unlike Dijkstra, Floyd-Warshall can handle negative edge weights, as long as there are no negative cycles.

## Introduction

Shortest path computation is a fundamental problem in computer science and graph theory, with applications in navigation systems, network routing, and logistics. Different algorithms are

designed to solve this problem with different trade-offs in speed, memory usage, and applicability depending on the structure of the graph.

This lab focuses on two widely used shortest path algorithms: Dijkstra's Algorithm and the Floyd-Warshall Algorithm. The goal is to implement these algorithms using dynamic programming principles, test them on sparse and dense graphs of varying sizes, and compare their performance. The lab also involves analyzing how the number of nodes influences execution time, presenting the results graphically, and drawing conclusions about the efficiency and suitability of each algorithm for different graph structures.

### Comparison Metric

The algorithms in this lab are compared based on two main metrics: execution time and graph density. Execution time measures how long each algorithm takes to compute shortest paths, recorded in milliseconds and averaged over multiple runs to reduce the effect of system noise. Graph density distinguishes between sparse graphs, where the number of edges E is close to V, and dense graphs, where E is close to $V^2$, directly influencing the performance of both algorithms. These metrics help evaluate the practical efficiency and suitability of each algorithm across different graph configurations.

### Input Format

The algorithms are tested on randomly generated weighted directed graphs with predefined node counts: 10, 50, 100, 200, and 500 nodes. For each node count, two graph variants are created: a sparse graph where the number of edges is approximately $1.5 \times V$, and a dense graph where the number of edges approaches $V \times (V - 1) / 2$. Edge weights are assigned as random integers in the range 1 to 100, ensuring all weights are positive and Dijkstra's algorithm remains applicable. Using these specific sizes and densities allows a clear comparison of how each algorithm scales with both increasing node count and increasing graph complexity.

## IMPLEMENTATION

### Dijkstra’s Algorithm

Dijkstra's Algorithm is a well-known graph algorithm used to find the shortest path from a single source node to all other nodes in a weighted graph. It follows a greedy approach combined with dynamic programming principles, always expanding the node with the smallest known distance.

It works by maintaining a priority queue of unvisited nodes, repeatedly selecting the closest unvisited node and relaxing its neighbors until all nodes have been processed.

*Complexity Analysis*

Time Complexity:

- Best Case: O((V + E) log V), when the graph is sparse and few edges need to be relaxed.
- Average Case: O((V + E) log V), when the graph has a typical distribution of edges.
- Worst Case: O((V + E) log V), when the graph is dense and all edges are processed.

Auxiliary Space: O(V), additional space is required for the distance array, the visited set, and the priority queue.

*Algorithm Description*

Dijkstra's Algorithm works by:

- Initialize: Set the distance to the source node as 0 and all other nodes as infinity. Insert the source into a min-heap priority queue.
- Select: Extract the node with the smallest distance from the priority queue.
- Relax: For each neighbor of the current node, calculate the new potential distance. If it is smaller than the known distance, update it and push the neighbor into the queue.
- Repeat: Continue until all nodes have been visited and the priority queue is empty.

*Implementation:*

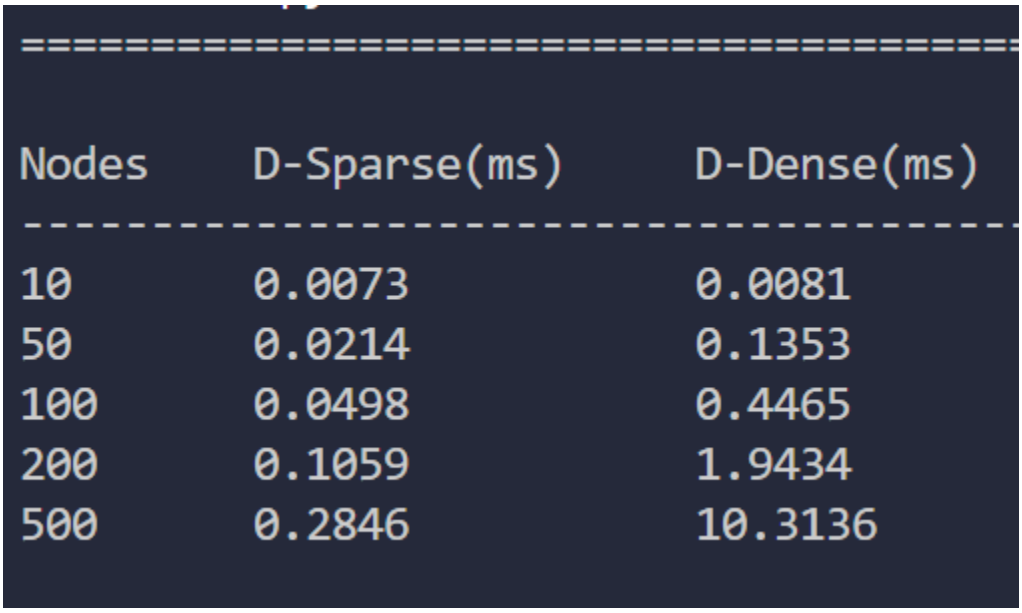
Lab4 > dijkstra.py > get_path

```python
import heapq


def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    previous = {node: None for node in graph}
    heap = [(0, start)]
    visited = set()

    while heap:
        current_dist, current_node = heapq.heappop(heap)

        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node]:
            if neighbor in visited:
                continue
            new_dist = current_dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                previous[neighbor] = current_node
                heapq.heappush(heap, (new_dist, neighbor))

    return distances, previous


def get_path(previous, start, end):
    path = []
    current = end
    while current is not None:
        path.append(current)
        current = previous[current]
    path.reverse()
    if not path or path[0] != start:
        return []
    return path
```

*Figure 2: Dijkstra's algorithm in Python*

Dijkstra's Algorithm was implemented in Python using a min-heap priority queue from the heapq module. The graph is represented as an adjacency list, where each node maps to a list of (neighbor, weight) pairs. The algorithm initializes all distances to infinity except the source, then iteratively relaxes edges by always processing the closest unvisited node first. A visited set prevents nodes from being processed more than once, ensuring correctness and efficiency.

*Results:*



| Nodes | D-Sparse(ms) | D-Dense(ms) |
|---|---|---|
| 10 | 0.0073 | 0.0081 |
| 50 | 0.0214 | 0.1353 |
| 100 | 0.0498 | 0.4465 |
| 200 | 0.1059 | 1.9434 |
| 500 | 0.2846 | 10.3136 |

*Figure 3: Results of Dijkstra's algorithm for inputs*

The algorithm was tested on randomly generated graphs with node counts of 10, 50, 100, 200, and 500. For each size, both a sparse graph ($E \approx 2V$) and a dense graph ($E \approx V^2$) were generated with random integer edge weights between 1 and 100. The observed runtimes are represented in Figure 3.

*Graph plot:*

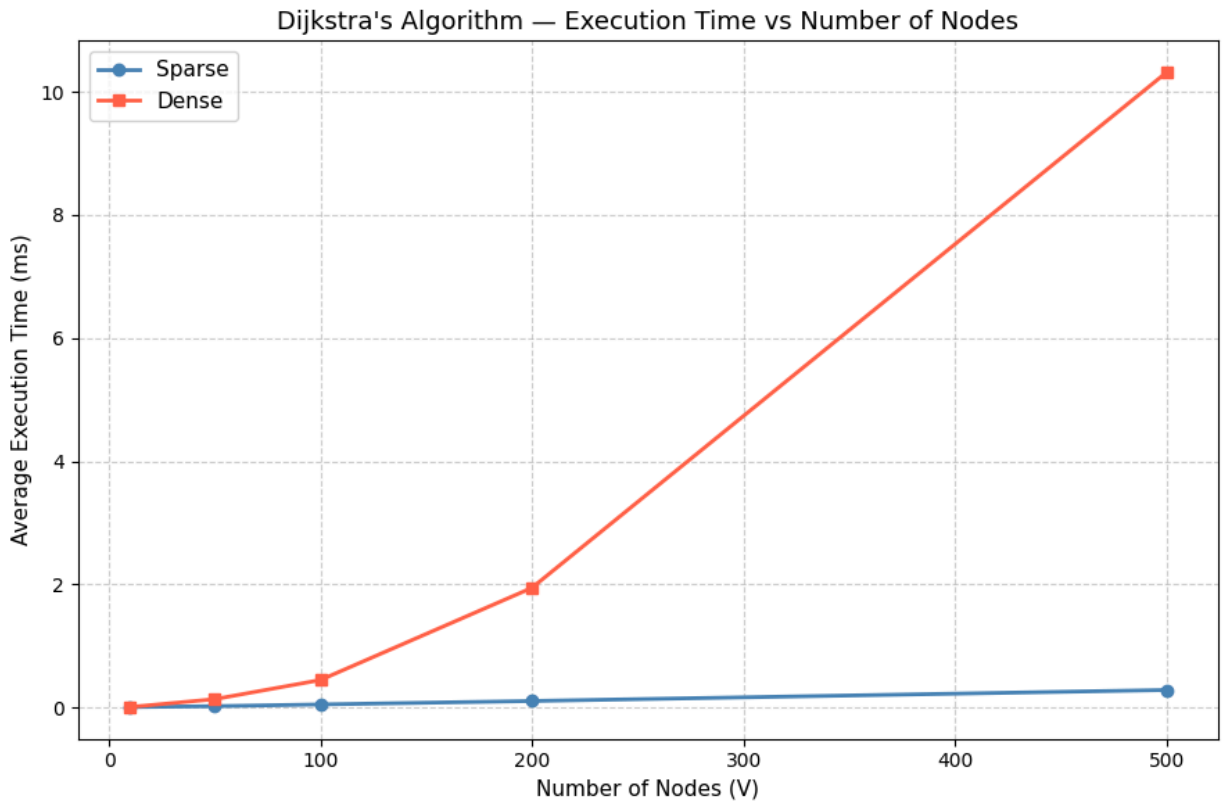



*Figure 4: Graph of Dijkstra's algorithm Runtime*

The plot shows a clear difference between sparse and dense graph performance. For sparse graphs, execution time grows slowly and nearly linearly, since the number of edges remains proportional to V. For dense graphs, execution time increases much more steeply as node count grows, reflecting the quadratic growth in edge count. This confirms that Dijkstra's Algorithm is highly efficient for sparse graphs and that graph density has a significant impact on performance.

### Floyd-Warshall Algorithm

The Floyd-Warshall Algorithm is a classic dynamic programming algorithm used to solve the all-pairs shortest path problem, finding the shortest paths between every pair of nodes in a weighted graph in a single execution. Unlike Dijkstra's Algorithm, it does not require a priority queue and works directly on an adjacency matrix, making it simple to implement. It supports negative edge weights as long as there are no negative cycles in the graph.

*Complexity Analysis*

Time Complexity:

- Best Case: $O(V^3)$, regardless of graph structure, all node triples are always evaluated.
- Average Case: $O(V^3)$, for any typical weighted graph with V nodes.
- Worst Case: $O(V^3)$, even for sparse graphs, all $V^3$ combinations are checked.

Auxiliary Space: O(V²), required to store the full distance matrix for all pairs of nodes.

*Algorithm Description*

Floyd-Warshall works by:

- Initialize: Build a distance matrix from the adjacency matrix, setting diagonal values to 0 and unknown paths to infinity.
- Iterate: For each intermediate node k, iterate over all pairs (i, j) and check whether routing through k produces a shorter path.
- Update: If the path through k is shorter, update the distance matrix entry for (i, j).
- Repeat: Continue for all V intermediate nodes until the matrix holds the shortest distances between every pair.

*Implementation:*

Lab4 > floyd_warshall.py > ...

```python
import copy

def floyd_warshall(matrix):
    n = len(matrix)
    dist = copy.deepcopy(matrix)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist


def has_negative_cycle(dist):
    """Check if any node has a negative distance to itself."""
    for i in range(len(dist)):
        if dist[i][i] < 0:
            return True
    return False
```

*Figure 5: Floyd-Warshall in Python*

The Floyd-Warshall Algorithm was implemented in Python using a three-nested-loop structure operating on a two-dimensional distance matrix. The matrix is initialized from the input adjacency matrix using a deep copy to avoid modifying the original graph. For each intermediate node k, all pairs (i, j) are evaluated and the matrix is updated if a shorter path through k is found.

The simplicity of the implementation reflects the straightforward nature of the dynamic programming recurrence it applies.

*Results:*

```
Lab4/main.py
================================================================================

Nodes     D-Sparse(ms)      D-Dense(ms)       FW-Sparse(ms)      FW-Dense(ms)
--------------------------------------------------------------------------------
10        0.0073            0.0081            0.0888             0.0697
50        0.0214            0.1353            7.6318             6.0200
100       0.0498            0.4465            51.2191            41.1693
200       0.1059            1.9434            419.1561           313.0293
500       0.2846            10.3136           7232.1616          5430.2508
```

*Figure 6: Floyd-Warshall runtime*

The algorithm was tested on randomly generated graphs with node counts of 10, 50, 100, 200, and 500, using both sparse ($E \approx 2V$) and dense ($E \approx V^2$) graph variants with random integer edge weights between 1 and 100. Testing was limited to 500 nodes due to the cubic time complexity making larger inputs impractical. The observed runtimes are represented in Figure 6.

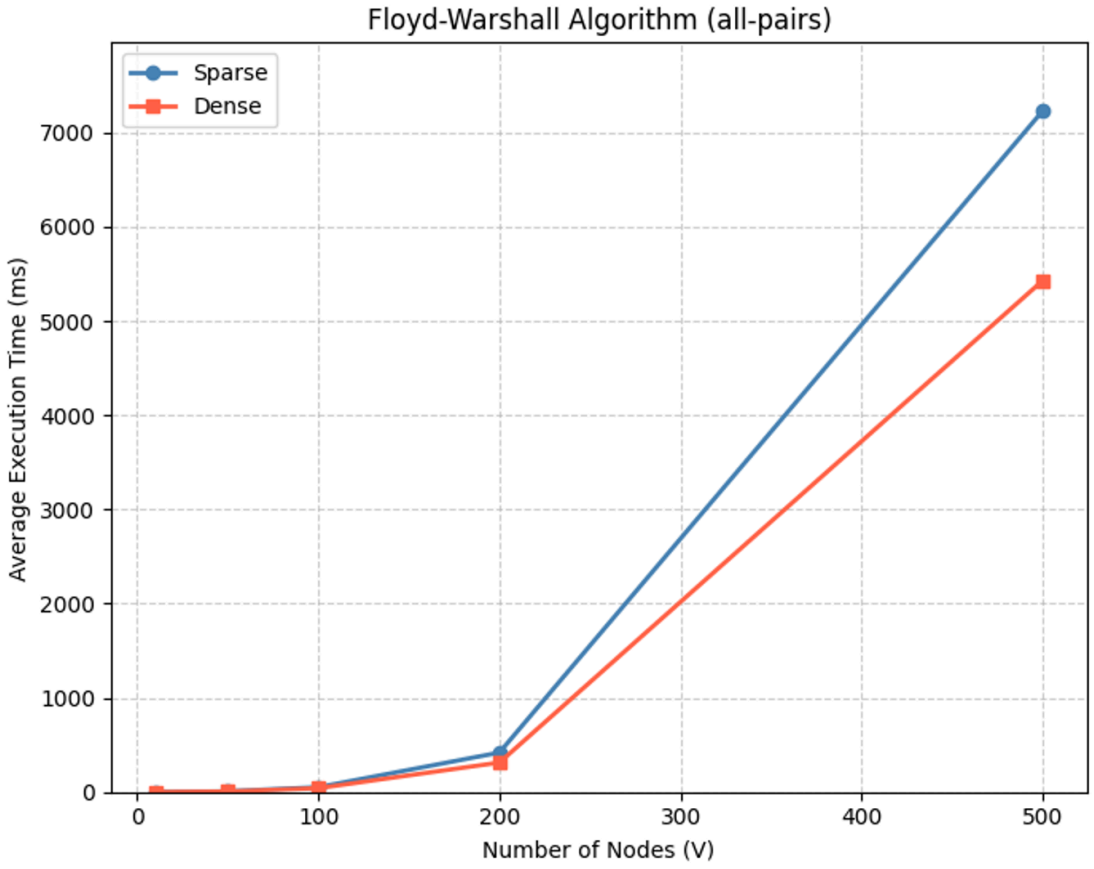



*Figure 7: Graph of Floyd-Warshall Runtime*

The plot shows that Floyd-Warshall's execution time grows steeply and consistently as node count increases, forming a curve that reflects its $O(V^3)$ complexity. Unlike Dijkstra's Algorithm, the difference between sparse and dense graphs is minimal, since Floyd-Warshall always

evaluates all $V^3$ node triples regardless of how many edges are present. This confirms that Floyd-Warshall is best suited for small to medium-sized dense graphs where all-pairs shortest paths are needed.

# CONCLUSION

In conclusion, Dijkstra's Algorithm and the Floyd-Warshall Algorithm address shortest path problems from two fundamentally different perspectives, and their practical usefulness depends directly on the structure of the problem being solved. Dijkstra's Algorithm is specifically designed for single-source shortest path computation and is most effective when the goal is to determine distances from one node to all others. Its use of a priority queue ensures that only the most promising nodes are explored at each step, making it highly efficient in sparse graphs and scalable to large inputs. In real-world applications such as road networks, routing systems, and large-scale graphs where queries typically originate from a single source, Dijkstra's Algorithm is generally the preferred choice due to its favorable time complexity and ability to avoid unnecessary computations.

In contrast, the Floyd-Warshall Algorithm is intended for all-pairs shortest path problems, where distances between every pair of nodes are required. Its strength lies in its simplicity and completeness: after a single execution, the shortest paths between all node pairs are known. This makes it particularly suitable for dense graphs or scenarios where repeated shortest path queries between arbitrary nodes are needed, such as network analysis, traffic flow optimization, or precomputed routing tables. Additionally, its ability to handle negative edge weights (in the absence of negative cycles) gives it an advantage in certain theoretical or specialized applications where Dijkstra's Algorithm cannot be applied.

However, this generality comes with a significant computational cost. The cubic time complexity of Floyd-Warshall makes it impractical for large graphs, as execution time increases rapidly even with moderate growth in the number of nodes. Unlike Dijkstra's Algorithm, it does not benefit from sparsity, since it evaluates all possible node triples regardless of edge count. As a result, its use is typically limited to small or medium-sized graphs where the overhead remains manageable and the need for all-pairs information justifies the cost.

The experimental results reinforce these observations. Dijkstra's Algorithm demonstrates strong performance on sparse graphs and remains efficient as graph size increases, while showing noticeable slowdown only in dense cases due to the higher number of edge relaxations. Floyd-Warshall, on the other hand, exhibits consistent but steep growth in runtime across both sparse and dense graphs, confirming that its performance is dominated entirely by the number of vertices rather than the number of edges.

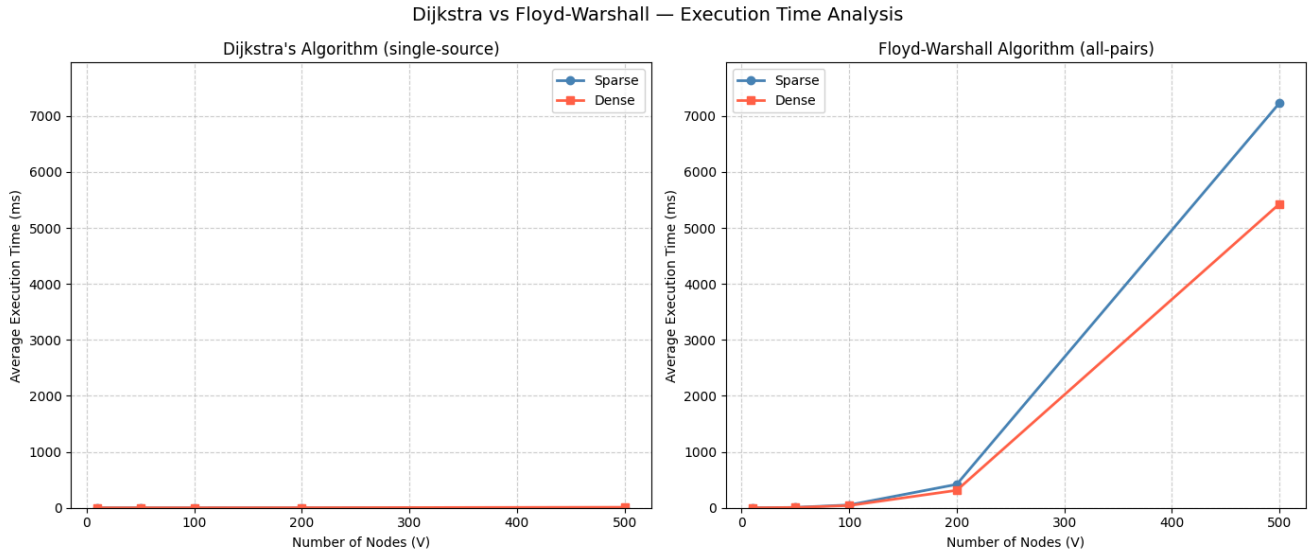



*Figure 8: Comparative runtime analysis of Dijkstra's and Floyd-Warshall algorithms on sparse and dense graphs*

Overall, the choice between the two algorithms should be guided by the problem requirements. If the task involves computing shortest paths from a single source in a large or sparse graph, Dijkstra's Algorithm is the more efficient and practical solution. If the task requires shortest paths between all pairs of nodes and the graph size is limited, Floyd-Warshall provides a straightforward and comprehensive approach.

# REFERENCES


1. https://github.com/kynexi/AA-labs/tree/main/Lab4
2. https://www.geeksforgeeks.org/dynamic-programming/
3. https://www.tutorialspoint.com/data_structures_algorithms/dynamic_programming.htm
4. https://www.codingninjas.com/codestudio/library/dijkstras-algorithm-vs-floydwarshalls-algorithm